GREEN QUANTA

AI-Assisted Quantum Learning Platform

TECHNICAL REPORT

Hack Apertus Online Hackathon • Own Project concept

Project author	Darshan Naik
Live application	https://green-quanta-ai-sigj.onrender.com
Source repository	https://github.com/darshannaik0915/green-quanta-ai
Technology stack	Python, FastAPI, JavaScript, HTML/CSS, Qiskit/Qiskit Aer, SQL/Supabase
Selected model	Apertus 1.5 8B (selected in submission form)

Purpose

This report describes Green Quanta's current platform, architecture and capabilities, plus a responsible technical plan for integrating Apertus 1.5 8B into its learning assistant. It distinguishes the deployed baseline from model-integration work that must be implemented and tested before it is claimed as completed.

Important status note: the deployed baseline and Qiskit circuit simulation are established project features. This report does not claim Apertus 1.5 is already connected to the live application. The Apertus-specific integration and evaluation below are proposed work unless you have independently completed and verified them.

1. Abstract and Problem Statement

Quantum computing can be difficult for beginners because abstract ideas—qubits, gates, superposition, measurement and circuits—are often taught separately from hands-on experimentation. Learners may understand definitions but struggle to see how gates change a state or how circuit operations relate to concepts.

Green Quanta is a browser-based learning platform that brings introductory lessons, quizzes, quantum circuit building, simulation, progress tracking and an AI-tutor experience into one environment. It aims to make learning more practical through short lessons and experimentation.

Objectives

• Offer a beginner-friendly introduction to quantum computing.
• Let learners build circuits with gates and inspect simulated outcomes.
• Reinforce concepts through quizzes and progress tracking.
• Explore Apertus 1.5 8B as an open model for educational tutoring, with transparent evaluation and clear disclosure of limitations.

2. Current Platform and Features

Feature	Purpose	Baseline status
Dashboard and lessons	Provide a central learning flow and structured quantum concepts.	Included in deployed platform
Circuit builder	Build gate sequences and explore simulation/state changes.	Qiskit/Qiskit Aer project capability
Quizzes and progress	Support practice and show learner progress.	Included in deployed platform
AI tutor interface	Provide conversational help with learning questions.	Tutor feature exists; Apertus backend connection must be verified separately

Intended users

Students, self-learners and educators who want an approachable introduction to quantum computing. The platform is an educational prototype and is not intended to replace a formal course, textbook or expert review.

3. System Architecture

The platform follows a browser-client and API-server structure. The frontend uses HTML, CSS and JavaScript. The backend uses Python and FastAPI. Quantum circuit operations use Qiskit and Qiskit Aer. The data layer uses Supabase/PostgreSQL, and the application is deployed on Render.

Layer	Responsibilities	Technology
Frontend	Login, dashboard, lessons, quizzes, progress, tutor UI and circuit-builder interactions.	HTML, CSS, JavaScript
Backend/API	Request handling and communication between UI, data and simulation components.	Python, FastAPI
Quantum simulation	Represent circuits and run simulations to explore gate sequences and results.	Qiskit, Qiskit Aer
Persistence	Store application data subject to configured schema and access controls.	Supabase/PostgreSQL, SQL
Hosting	Make the web application available online.	Render
LLM tutor adapter	Proposed backend connection that sends learner questions to Apertus 1.5 8B and returns generated explanations.	Apertus 1.5 8B; integration to be tested

Proposed request flow for an Apertus-powered tutor

1. Learner enters a question in the tutor interface.
2. The frontend sends it to a backend endpoint over HTTPS.
3. The backend validates input and optionally adds a short, approved lesson excerpt as context.
4. The backend calls the configured Apertus 1.5 8B inference endpoint using a server-side credential.
5. The backend returns the answer and handles provider errors or timeouts.
6. The learner receives an explanation and is encouraged to verify important technical claims.

Security

Credentials must remain in server-side environment variables, never in frontend JavaScript or a public repository. Use input limits, timeouts and basic rate limiting. Minimize learner data and be transparent about any stored conversation history.

4. Apertus 1.5 8B Integration Plan

The submission form identifies Apertus 1.5 8B as the selected model. The proposed role is to power the educational tutor: explain quantum concepts, walk through simple gate sequences, clarify quiz feedback and offer hints without immediately giving away answers.

Target tutor behaviour

- Explain one concept at a time in beginner-friendly language.
- Define technical terms and use small examples.
- For circuit questions, use circuit details supplied by the application instead of inventing gate sequences.
- Offer hints before full quiz answers.
- Acknowledge uncertainty and recommend checking lesson material when appropriate.

Implementation and evidence checklist

Step	Task	Evidence to record
1	Select a supported Apertus 1.5 8B inference provider or deployment.	Model/provider details and a successful test request.
2	Create a backend-only adapter using environment-based credentials.	Successful response; no secrets exposed in browser.
3	Connect tutor UI to the backend endpoint.	Screenshot or recording of a real model response.
4	Add validation, timeouts, error handling and basic request limits.	Tests for empty input, long input, timeout and provider errors.
5	Evaluate representative quantum-learning questions.	Test prompts with recorded quality observations.

Suggested evaluation prompts

1. Explain a qubit to a first-year student.
2. What does a Hadamard gate do conceptually?
3. Explain the difference between a quantum state and a measurement result.
4. Explain the effect of applying H then X to a single qubit without inventing an unsimulated measurement outcome.
5. If uncertain, does the model say so and suggest verification?

No benchmark scores or measured performance are claimed here. Add results only after running and recording actual tests.

5. Validation, Limitations and Future Work

Validation checklist

• Confirm the deployed website loads and main screens are reachable.
• Test a small known circuit and compare simulation results with expected behaviour.
• Test login, API authorization and database operations with a non-sensitive test account.
• If Apertus is integrated, record the exact model/provider, date, configuration and sample prompts.
• Evaluate tutor answers for correctness, clarity, relevance, uncertainty and latency.
• Confirm credentials do not appear in frontend code, logs or this report.

Limitations

• A language model may produce plausible but incorrect explanations; tutor responses are learning assistance, not authoritative answers.
• Simulations do not represent every physical-device effect unless those effects are explicitly modelled.
• Tutor quality depends on the inference provider, prompts and context. This report does not verify that Apertus is connected to the live application.
• The platform is an educational prototype, not a validated curriculum or research benchmark.

Future work

1. Complete and verify Apertus 1.5 8B tutor integration.
2. Add a curated quantum-concept reference set for grounded explanations.
3. Add learner-appropriate hints and progression while minimizing personal-data collection.
4. Expand circuit examples to include common beginner algorithms and step-by-step state inspection.
5. Publish reproducible tests and document observed limitations.

6. Conclusion

Green Quanta combines lessons, quizzes, progress tracking and hands-on quantum circuit experimentation in one web application. Its FastAPI and Qiskit-based architecture provides a foundation for an AI-assisted learning experience. Integrating Apertus 1.5 8B into the tutor could demonstrate an open language model applied to technical education, provided the integration is completed, tested and reported transparently.

Project links

Live demo: https://green-quanta-ai-sigj.onrender.com
GitHub: https://github.com/darshannaik0915/green-quanta-ai
Apertus documentation: https://www.apertus-ai.org/pages/documentation/
Hackathon: https://hackapertus.ch/online-hack

Update this report after the Apertus integration and evaluation are completed. Do not describe proposed work as completed implementation.